

A Formal Analysis of the Logarithmic Median Search

Abstract—This paper presents a rigorous breakdown of the logarithmic time complexity median search algorithm over two sorted arrays, demonstrating its superiority over trivial linear merges.



1 Introduction

The problem of finding the median of two sorted arrays is a fundamental algorithmic challenge with applications in distributed databases and order statistics. Given two sorted arrays, A of size m and B of size n , the trivial solution involves merging the arrays into a single sorted sequence of size $m + n$ and extracting the median in $\mathcal{O}(m + n)$ time. However, in latency-critical systems, a linear time bound is sub-optimal. This paper presents a formal analysis of a binary search partition algorithm that achieves the theoretical lower bound of $\mathcal{O}(\log(\min(m, n)))$ time complexity.

2 Problem Definition

Let $A = \langle a_1, a_2, \dots, a_m \rangle$ and $B = \langle b_1, b_2, \dots, b_n \rangle$ be two monotonically non-decreasing sequences. We define the merged sequence $C = A \cup B$ such that C is sorted and $|C| = m + n$. The objective is to identify the median value of C without explicitly constructing C .

The median is formally defined as the value separating the higher half from the lower half of the dataset. If $m + n$ is odd, the median is the value at index $(m + n + 1)/2$. If $m + n$ is even, it is the arithmetic mean of the values at indices $(m + n)/2$ and $(m + n)/2 + 1$.

3 The Logarithmic Partition Strategy

To achieve sub-linear time complexity, we employ a divide-and-conquer strategy utilizing the properties of sorted sequences. The core insight is that finding the median is equivalent to dividing the combined multi-set into two disjoint subsets, L (left) and R (right), satisfying two conditions:

- 1) $|L| = |R|$ (or $|L| = |R| + 1$ for odd lengths)
- 2) $\max(L) \leq \min(R)$

We achieve this by making a partition in array A at index i and a corresponding partition in array B at index j .

3.1 Partition Invariants

Let the total length be $N = m + n$. The partition index i in array A uniquely determines the partition index j in array B to maintain the size invariant of subset L :

$$j = \left\lfloor \frac{m + n + 1}{2} \right\rfloor - i \quad (1)$$

By performing a binary search on the smaller array (assume $|A| \leq |B|$ without loss of generality), we can find the optimal partition index $i \in [0, m]$.

3.2 Validation of Partition

For a given i and derived j , the partition is valid if and only if the cross-boundary conditions hold:

$$A[i - 1] \leq B[j] \quad (2)$$

$$B[j - 1] \leq A[i] \quad (3)$$

If $A[i - 1] > B[j]$, the partition in A is too far to the right, and we must adjust our binary search boundary leftwards. Conversely, if $B[j - 1] > A[i]$, the partition is too far to the left.

4 Complexity Analysis

Theorem 1. The proposed partition algorithm determines the median in $\mathcal{O}(\log(\min(m, n)))$ time and $\mathcal{O}(1)$ auxiliary space.

Proof. Let $m = |A|$ and $n = |B|$. By ensuring we only perform the binary search over the smaller array A , the search space is restricted to $[0, m]$. In each iteration of the binary search, the search space is halved. Therefore, the maximum number of iterations is strictly bounded by $\lceil \log_2(m) \rceil$. Since all operations within the loop (index arithmetic, boundary condition checks, and median calculation) execute in constant time $\mathcal{O}(1)$, the overall time complexity is $\mathcal{O}(\log(\min(m, n)))$. The space complexity is $\mathcal{O}(1)$ as no dynamic allocations or recursive stack frames are required. ■

5 Conclusion

The logarithmic partition method demonstrates optimal asymptotic behavior for the two-array median problem. By leveraging the inherent order of the input sequences and establishing strict mathematical invariants for cross-array partitioning, we eliminate the need for linear merging, yielding a highly performant algorithm suitable for large-scale data processing.